

Banco de Dados

Prof^a Angelica Nakagawa Lima
angelica.lima@ibta.edu.br

Manipulando dados

COMPARATIVO DE TIPOS DE DADOS: ORACLE VS MYSQL

ORACLE
DATABASE

MySQL DATABASE

NUMBER (P, S)
(ex: NUMBER, NUMBER(10,2))

NUMÉRICOS

TINYINT | **SMALLINT** | **INT** | **BIGINT** | **DECIMAL(P,D)**
Descrição de número/ínteros | Descrição de inteiro de pequena | Descrição de inteiro de configuração | Descrição de números | Descrição da semelhança decimal

VARCHAR2(N) | **CHAR(N)** | **CLOB**

TEXTO

VARCHAR(N) | **CHAR(N)** | **TEXT** | **LONGTEXT**

DATE | **TIMESTAMP** | **INTERVAL**
(includes time)

DATA & HORA

DATE | **TIME** | **DATETIME** | **TIMESTAMP** | **YEAR**

RAW(N) | **BLOB** | **BFILE**

BINÁRIOS

BINARY(N) | **VARBINARY(N)**
TINYBLOB | **BLOB** | **LOB**

BOOLEAN | **XMLTYPE**
(in SQL)

OUTROS

BOOLEAN | **JSON** | **ENUM** | **SET**
(synonym for TINYINT(1))





PRINCIPAIS COMANDOS DDL

Data Definition Language (Linguagem de Definição de Dados)

1 CREATE

Cria uma nova tabela ou objeto no banco de dados.

```
CREATE TABLE nome_tabela (  
  coluna1 TIPO_DADO restricoes,  
  coluna2 TIPO_DADO,  
  ...  
);
```

2 ALTER

Modifica a estrutura de uma tabela já existente (adiciona ou remove colunas).

```
-- Adicionar coluna  
ALTER TABLE nome_tabela  
ADD nova_coluna TIPO_DADO;  
  
-- Remover coluna  
ALTER TABLE nome_tabela
```

3 DROP

Exclui permanentemente uma tabela, incluindo sua estrutura e todos os dados.

```
DROP TABLE nome_tabela;
```

4 TRUNCATE

Remove todos os registros da tabela rapidamente, mas **mantém a estrutura**.

```
TRUNCATE TABLE nome_tabela;
```

5 RENAME

Altera o nome de uma tabela já existente no banco de dados.

```
RENAME TABLE nome_antigo  
TO nome_novo;
```

Exercício

Criar as tabelas conforme estrutura abaixo:

Tabela: **Fornecedor**

cd_fornecedor	number (4),
nm_fornecedor	varchar2 (50),
ds_endereco	varchar2 (70),
nr_fone	number (12),
dt_nascimento	date,
ie_sexo	char (1),
ie_fisica_juridica	char (1),
cd_situacao	number (3)

Tabela: **Pais**

cd_pais	number(2),
nm_pais	varchar2 (45),
ds_nacionalidade	varchar2(45)

Tabela: **Estado**

sg_estado	char(2),
nm_estado	varchar2 (35)

Tabela: **Cliente**

cd_cliente	number (4),
nm_cliente	varchar2 (50),
ds_endereco	varchar2 (70),
cd_municipio	number (5),
sg_estado	char (2),
nr_cep	varchar2 (8),
nr_ddd	number (3),
nr_fone	number(7),
ie_sexo	char(1)

Tabela: **Municipio**

cd_municipio	number(5),
sg_estado	char (2),
nm_municipio	varchar2 (80)

Tabela: **Estado_Civil**

cd_estado_civil	number (1),
ds_estado_civil	varchar2 (10)

Exercício

1. Alterar as tabelas **Fornecedor**, **Cliente** e **Estado_Civil** tornando os códigos chave primária em cada uma delas;
2. Acrescentar na tabela **Cliente** o campo cd_estado_civil, que deverá ser um código existente na tabela **Estado_Civil**.
3. Acrescentar na tabela **Fornecedor** os campos cd_pais, sg_estado e cd_municipio, numérico de 2 posições, char com 2 posições e numérico com 5 posições, respectivamente. Na tabela **Cliente** incluir o campo cd_pais numérico com 2 posições. Este campo também deve ser incluído nas tabelas **Estado** e **Municipio**.

Inserção de Dados - INSERT

- Para inserir registros em uma tabela, utilizados o comando INSERT, respeitando a sintaxe abaixo:

```
INSERT INTO nome_da_tabela  
[ ( nome_da_coluna [ , nome_da_coluna ] ) ]  
VALUES  
( valor_da_coluna [ , valor_da_coluna ] )
```

- Valores alfanuméricos e datas devem sempre estar entre **aspas simples (' ')**

OBS: O que está entre colchetes ([]) é opcional

Inserção de Dados - Exemplos

- Antes de fazer os exemplos precisamos criar a tabela abaixo:

```
CREATE TABLE pais  
  (cd_pais number (2),  
   nm_pais char (45),  
   ds_nacionalidade char(35),  
   constraint pais_cd_pais_pk primary key  
   (cd_pais),  
   constraint pais_nm_pais_un unique  
   (nm_pais));
```


Inserção de Dados - Exemplos

- Para incluir dados em uma tabela podemos ou não incluir valores em todas as colunas (caso não sejam obrigatórias).

```
INSERT INTO pais (cd_pais, nm_pais)  
VALUES (1,'Brasil');
```

- A relação de colunas da tabela poderá ser omitida quando forem incluídos valores para todas as colunas e na ordem em que foram definidas.

```
INSERT INTO pais  
VALUES (4,'Chile', 'Chilena');
```

Inserção de Dados - Exemplos

- Ao inserir valores na tabela podemos alterar a ordem original das colunas, mas neste caso devemos indicar a relação de colunas na ordem em que vamos inserir os valores.

```
INSERT INTO pais (nm_pais, ds_nacionalidade,  
cd_pais)  
VALUES ('EUA', 'Americana', 3);
```

Table PAIS criado.

Nome	Nulo?	Tipo
CD_PAIS	NOT NULL	NUMBER(2)
NM_PAIS		CHAR(45)
DS_NACIONALIDADE		CHAR(35)

Inserção de Dados - Exemplos

- Crie as tabelas a seguir:

```
CREATE TABLE Disciplina(  
  cd_disciplina NUMBER(3),  
  ds_nome VARCHAR2(30) NOT NULL,  
  CONSTRAINT disciplina_pk PRIMARY  
  KEY(cd_disciplina) );
```

Inserção de Dados - Exemplos

- Crie as tabelas a seguir:

```
CREATE TABLE Matricula(  
  nr_rgm VARCHAR2(10),  
  cd_disciplina NUMBER(3) NOT NULL,  
  ds_situacao VARCHAR2(10),  
  vl_media NUMBER(4),  
  CONSTRAINT matricula_pk PRIMARY KEY(nr_rgm),  
  CONSTRAINT matricula_cd_disciplina_fk FOREIGN KEY  
  (cd_disciplina) REFERENCES Disciplina(cd_disciplina),  
  CONSTRAINT aluno_vl_media_ck CHECK(vl_media >=0  
  AND vl_media <=10 )  
);
```

Inserção de Dados - Exemplos

- Crie e execute os INSERTs abaixo:

```
INSERT INTO Matricula VALUES ('1234567', 2, null, 7.5);  
INSERT INTO Matricula VALUES ('1234567', 2, null, 10); INSERT  
INTO Matricula VALUES ('1111111', 2, null, 5.2); INSERT INTO  
Matricula VALUES ('2222222', 2, null, 6); INSERT INTO Matricula  
VALUES ('3333333', 2, null, 4.5); INSERT INTO Matricula  
VALUES ('5555555', 2, null, 11);
```

```
INSERT INTO Disciplina VALUES (1, 'Banco de Dados');  
INSERT INTO Disciplina VALUES (2, 'Aplicações para Internet');
```

Acontecem erros de violação de restrição

Alteração de Dados - UPDATE

- Para modificar os dados armazenados utilizamos o comando UPDATE, respeitando a sintaxe abaixo:

```
UPDATE Tabela  
  SET Coluna = Valor  
  WHERE Condição
```

- Valores alfanuméricos e datas devem sempre estar entre **aspas simples (' ')**

Alteração de Dados - Exemplos

- Podemos alterar os valores de mais de um campo ou não.

```
UPDATE Pais SET  
    nm_pais='Portugal',  
    ds_nacionalidade='Portug'  
WHERE cd_pais = 3;
```

```
UPDATE Pais SET  
    ds_nacionalidade='Portuguesa'  
WHERE cd_pais = 3;
```

Alteração de Dados - Exemplos

- Alterando o valor do campo ds_situacao:

```
UPDATE Matricula SET ds_situacao = 'aprovado'  
WHERE vl_media >= 6;
```

```
UPDATE Matricula SET ds_situacao = 'af'  
WHERE vl_media < 6 AND vl_media >= 1;
```

```
UPDATE Matricula SET ds_situacao = 'reprovado'  
WHERE vl_media < 1;
```


Exclusão de Dados - DELETE

- Para excluir os dados armazenados utilizamos o comando DELETE, respeitando a sintaxe abaixo:

```
DELETE FROM Tabela  
WHERE Condição
```

```
DELETE FROM Pais  
WHERE cd_pais = 3;
```

- Nunca omita a cláusula WHERE. No caso da omissão, todos os registros da tabela serão apagados.

Consultando dados - SELECT

- O comando SELECT permite a seleção e manipulação, para visualização das informações armazenadas no banco de dados.
- Veja a sintaxe básica:

```
SELECT col1, col2, coln  
FROM nome da tabela  
WHERE condição
```

Consultando datos - Ejemplos

- Podemos seleccionar ...

```
SELECT * FROM departments;
```

	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
1	10	Administration	200	1700
2	20	Marketing	201	1800
3	30	Purchasing	114	1700
4	40	Human Resources	203	2400
5	50	Shipping	121	1500
6	60	IT	103	1400
7	70	Public Relations	204	2700
8	80	Sales	145	2500
9	90	Executive	100	1700
10	100	Finance	108	1700
11	110	Accounting	205	1700
12	120	Treasury	(null)	1700
13	130	Corporate Tax	(null)	1700
14	140	Control And Credit	(null)	1700
15	150	Shareholder Services	(null)	1700

Consultando datos - Ejemplos

- Podemos seleccionar ...

SELECT first_name, email, hire_date **FROM**
employees;

	FIRST_NAME	EMAIL	HIRE_DATE
1	Steven	SKING	17/06/87
2	Neena	NKOCHHAR	21/09/89
3	Lex	LDEHAAN	13/01/93
4	Alexander	AHUNOLD	03/01/90
5	Bruce	BERNST	21/05/91
6	David	DAUSTIN	25/06/97
7	Valli	VPATABAL	05/02/98
8	Diana	DLORENTZ	07/02/99
9	Nancy	NGREENBE	17/08/94
10	Daniel	DFAVIET	16/08/94
11	John	JCHEN	28/09/97
12	Ismael	ISCIARRA	30/09/97
13	Jose Manuel	JMURMAN	07/03/98
14	Luis	LPOPP	07/12/99
15	Den	DRAPHEAL	07/12/94
16	Alexander	AKHOO	18/05/95
17	Shelli	SBAIDA	24/12/97

Consultando datos - Ejemplos

- Podemos seleccionar ...

```
SELECT first_name, email , salary, salary*12  
FROM employees;
```

	FIRST_NAME	EMAIL	SALARY	SALARY*12
1	Steven	SKING	24000	288000
2	Neena	NKOCHHAR	17000	204000
3	Lex	LDEHAAN	17000	204000
4	Alexander	AHUNOLD	9000	108000
5	Bruce	BERNST	6000	72000
6	David	DAUSTIN	4800	57600
7	Valli	VPATABAL	4800	57600
8	Diana	DLORENTZ	4200	50400
9	Nancy	NGREENBE	12000	144000
10	Daniel	DFAVIET	9000	108000
11	John	JCHEN	8200	98400
12	Ismael	ISCIARRA	7700	92400
13	Jose Manuel	JMURMAN	7800	93600
14	Luis	LPOPP	6900	82800
15	Den	DRAPHEAL	11000	132000
16	Alexander	AKHOO	3100	37200

Consultando datos - Exemplos

- Podemos seleccionar ...

```
SELECT first_name, email, salary "Salário",  
salary*12 "Salario Anual" FROM employees;
```

Alias
(apelido)

Alias
(apelido)

	FIRST_NAME	EMAIL	Salário	Salario Anual
1	Steven	SKING	24000	288000
2	Neena	NKOCHHAR	17000	204000
3	Lex	LDEHAAN	17000	204000
4	Alexander	AHUNOLD	9000	108000
5	Bruce	BERNST	6000	72000
6	David	DAUSTIN	4800	57600
7	Valli	VPATABAL	4800	57600
8	Diana	DLORENTZ	4200	50400
9	Nancy	NGREENBE	12000	144000
10	Daniel	DFAVIET	9000	108000
11	John	JCHEN	8200	98400
12	Ismael	ISCIARRA	7700	92400
13	Jose Manuel	JMURMAN	7800	93600
14	Luis	LPOPP	6900	82800
15	Den	DRAPHEAL	11000	132000
16	Alexander	AKHOO	3100	37200
17	Shelli	SBAIDA	2900	34800

Consultando dados - Exemplos

- Podemos selecionar ...

Concatenação

```
SELECT first_name, email, 'Salário: ' || salary "Salário",  
salary*12 "Salario Anual" FROM employees;
```

Alias
(apelido)

FIRST_NAME	EMAIL	Salário	Salario Anual
1 Steven	SKING	Salário: 24000	288000
2 Neena	NKOCHHAR	Salário: 17000	204000
3 Lex	LDEHAAN	Salário: 17000	204000
4 Alexander	AHUNOLD	Salário: 9000	108000
5 Bruce	BERNST	Salário: 6000	72000
6 David	DAUSTIN	Salário: 4800	57600
7 Valli	VPATABAL	Salário: 4800	57600
8 Diana	DLORENTZ	Salário: 4200	50400
9 Nancy	NGREENBE	Salário: 12000	144000
10 Daniel	DFAVIET	Salário: 9000	108000
11 John	JCHEN	Salário: 8200	98400
12 Ismael	ISCIARRA	Salário: 7700	92400
13 Jose Manuel	JMURMAN	Salário: 7800	93600
14 Luis	LPOPP	Salário: 6900	82800
15 Den	DRAPHEAL	Salário: 11000	132000

Consultando dados - DISTINCT

- A exibição *default* de consultas são todas as linhas, incluindo as linhas duplicadas;
- Para eliminar valores duplicados no resultado, incluimos o **DISTINCT** qualificador no comando **SELECT**.

```
SELECT department_id  
FROM employees  
ORDER BY department_id;
```

	DEPARTMENT_ID
1	10
2	20
3	20
4	30
5	30
6	30
7	30
8	30
9	30
10	40
11	50
12	50
13	50

```
SELECT DISTINCT department_id  
FROM employees  
ORDER BY department_id;
```

	DEPARTME...
1	10
2	20
3	30
4	40
5	50
6	60
7	70
8	80
9	90
10	100
11	110
12	(null)

Consultando dados - Operadores

- Expressões Aritméticas podem compor nome de colunas, valores numéricos constantes e operadores aritméticos:

Operadores	Descrição
+	Adição
-	Subtração
*	Multiplicação
/	Divisão

```
SELECT first_name, email  
, salary, salary*12  
FROM employees;
```

	FIRST_NAME	EMAIL	SALARY	SALARY*12
1	Steven	SKING	24000	288000
2	Neena	NKOCHHAR	17000	204000
3	Lex	LDEHAAN	17000	204000
4	Alexander	AHUNOLD	9000	108000
5	Bruce	BERNST	6000	72000
6	David	DAUSTIN	4800	57600
7	Valli	VPATABAL	4800	57600
8	Diana	DLORENTZ	4200	50400
9	Nancy	NGREENBE	12000	144000
10	Daniel	DFAVIET	9000	108000
11	John	JCHEN	8200	98400
12	Ismael	ISCIARRA	7700	92400
13	Jose Manuel	JMURMAN	7800	93600
14	Luis	LPOPP	6900	82800
15	Den	DRAPHEAL	11000	132000
16	Alexander	AKHOO	3100	37200

Consultando dados - Operadores

- A Cláusula **WHERE** indica condição para um **SELECT**, no qual pode ter os seguintes operadores:

Operadores	Descrição
=	Igualdade
<>	Diferente
>	Maior
<	Menor
>=	Maior ou igual
<=	Menor ou igual

```
SELECT first_name, email ,  
salary, salary*12  
FROM employees  
WHERE salary < 9000;
```

	FIRST_NAME	EMAIL	SALARY	SALARY*12
1	James	JLANDRY	2400	28800
2	Steven	SMARKLE	2200	26400
3	TJ	TJOLSON	2100	25200
4	Ki	KGEE	2400	28800
5	Hazel	HPHILTAN	2200	26400

Criação de tabela a partir de uma consulta

- Este comando cria uma tabela (estrutura e dados) baseado em um comando **SELECT**.

```
CREATE TABLE Espelho_Pais  
AS SELECT * FROM Pais;
```

`select * from pais;`

Saída do Script * Resultado da Consulta *

Todas as Linhas Extraídas: 4 em 0,047 segundos

	CD_PAIS	NM_PAIS	DS_NACIONALIDADE
1	1	Brasil	... (null)
2	2	França	... (null)
3	3	EUA	... (null)
4	4	Chile	... (null)

`select * from Espelho_Pais;`

Saída do Script * Resultado da Consulta *

Todas as Linhas Extraídas: 4 em 0,046 segundos

	CD_PAIS	NM_PAIS	DS_NACIONALIDADE
1	1	Brasil	... (null)
2	2	França	... (null)
3	3	EUA	... (null)
4	4	Chile	... (null)

Agradecimentos

Slides baseados nas aulas da prof. Cristiane - Unicsul